INFO 7610 Topic Nat Lang Engr Meth/Tools

Natural Language Processing Tasks

Feifei Pan

Agenda

- Assignment 1
- Natural Language Processing Tasks
  - Revisit some tasks we discussed last time…
  - Summarization
  - Machine Translation
  - Question Answering
  - Advanced tasks and Emerging Topics

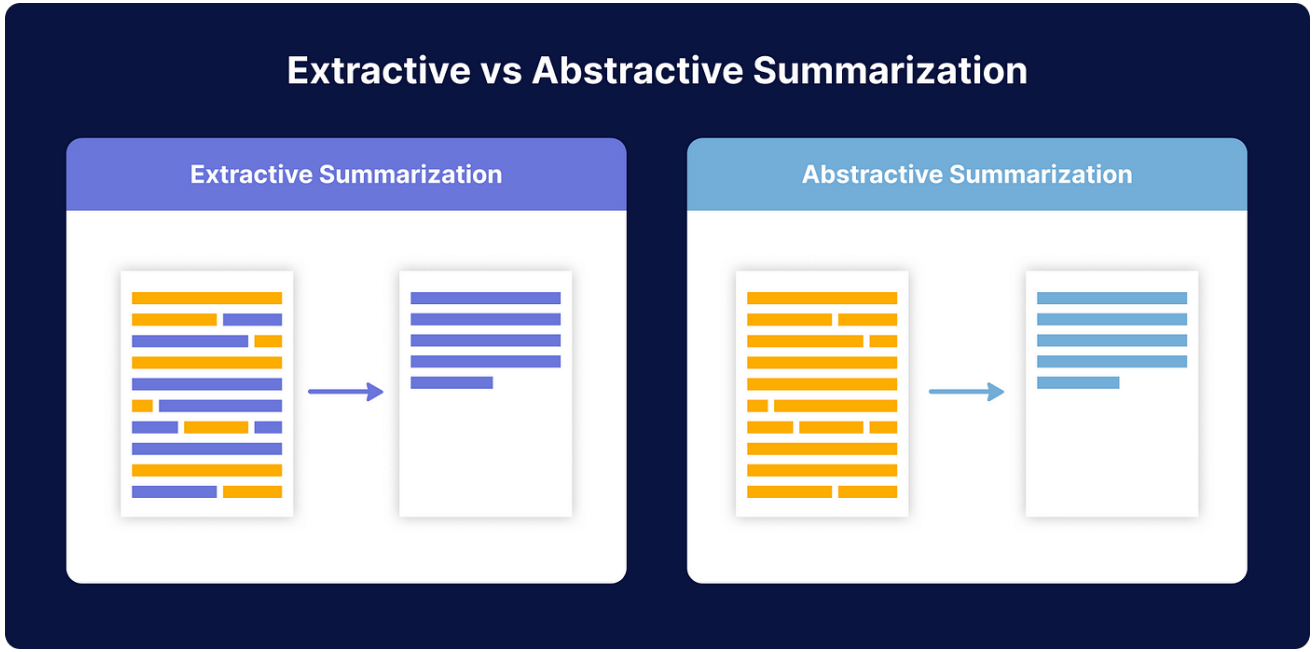
NLP Tasks Recap

- Stats-based Language Model
  - Bi-Gram model example
- Tokenization
  - BPE—low lower lowest
  - SentencePiece
  - WordPiece
- Text classification
- Name Entity Recognition

Aspect	BPE	SentencePiece	WordPiece
Advantages	Simple and intuitive; works well for many languages.	Language-independent; handles raw text directly.	High-quality segmentation; integrates well with models like BERT.
Disadvantages	Sensitive to language preprocessing; may struggle with non-spaced languages.	Adds special tokens for spaces, which may impact interpretability.	Requires pre-tokenized input, less flexible for non-spaced languages.

Summarization

- **Text Summarization** is the task of generating a concise and coherent summary from a larger body of text while preserving the key information. Summarization can be broadly classified into two categories:
  - **Extractive Summarization**: Selects and extracts key phrases or sentences directly from the source text to form a summary.
  - **Abstractive Summarization**: Generates new sentences that capture the meaning of the original text, often using paraphrasing and rephrasing.
- Summarization has applications in many domains
  - news articles, academic papers, customer support, and document summarization for legal and healthcare domains.



Extractive Summarization Model

- Traditional Methods for Extractive Summarization
  - **TF-IDF (Term Frequency-Inverse Document Frequency)**: a statistical measure that evaluates the importance of words in a document. Sentences with the highest aggregate TF-IDF scores are selected as the most important for the summary.
  - **Graph-based Methods (e.g., TextRank)**: an algorithm based on the PageRank algorithm used by Google to rank web pages. It builds a graph where sentences are nodes and the edges between them represent similarity. Sentences with the highest centrality scores are selected for the summary.
  - **Latent Semantic Analysis (LSA)**: uses singular value decomposition (SVD) to reduce dimensionality and capture the latent structure of the text. Sentences are ranked based on their relevance to the underlying concepts in the document.
  - **Clustering Algorithms**: Techniques like **k-means clustering** can be used to group similar sentences together, and one representative sentence from each cluster can be selected to form the summary.

TF-IDF Idea

- The general idea behind **extractive summarization** is:

1. **Extract** a subset of sentences from the original document.
2. **Rank** the sentences based on their importance in representing the main content of the document.
3. **Select** the top-ranked sentences to form a summary.

$$TF = \frac{Number\ of\ times\ a\ word\ "X"\ appears\ in\ a\ Document}{Number\ of\ words\ present\ in\ a\ Document}$$

$$IDF = log\left(\frac{Number\ of\ Documents\ present\ in\ a\ Corpus}{Number\ of\ Documents\ where\ word\ "X"\ has\ appeared}\right)$$

$$TF\ IDF\ =\ TF * IDF$$

Extractive Summarization with TF-IDF

- **Tokenization**: Split the text into sentences→terms
- **TF-IDF Computation**: Calculate the TF-IDF score for each term (word) in each sentence.
- **Sentence Scoring**: For each sentence, compute a score by summing up the TF-IDF scores of the words in that sentence.
- **Sentence Selection**: Rank the sentences based on their scores and extract the top n sentences to form the summary.
- In practice
  - import nltk
  - from sklearn.feature_extraction.text import TfidfVectorizer

```
# Tokenize document into sentences
sentences = nltk.sent_tokenize(document)

# Initialize TF-IDF Vectorizer
vectorizer = TfidfVectorizer(stop_words='english')

# Fit and transform the sentences to get the TF-IDF matr
tfidf_matrix = vectorizer.fit_transform(sentences)

# Compute the sum of TF-IDF values for each sentence
sentence_scores = np.array(tfidf_matrix.sum(axis=1)).fla
```

$$\text{BM25}(t,d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t,d)\cdot(k_1+1)}{f(t,d)+k_1\cdot(1-b+b\cdot\frac{|d|}{\text{avgdl}})}$$

where:

- $f(t,d)$ is the frequency of t in d,
- $|d|$ is the length of document d,
- avgdl is the average document length in the collectic
- k_1 and b are hyperparameters that control the weight

Pros and Cons

- **Advantages of TF-IDF for Extractive Summarization**
  - **Simplicity**: TF-IDF is easy to implement and computationally efficient.
  - **Effectiveness**: It works well for documents with clearly defined important sentences.
- **Limitations**
  - **Context Ignorance**: TF-IDF doesn't capture contextual meaning or relationships between sentences (e.g., it doesn't consider the semantic meaning of a sentence).
  - **No Cohesion**: The summary might contain sentences that are relevant individually but lack cohesion when combined.

Extractive Summarization Model

- **BERT-based Models (e.g., BERTSUM)**
  - **BERTSUM** is one such model that fine-tunes BERT for extractive summarization. It processes the text to generate embeddings for each sentence and ranks sentences based on their relevance to the document.
  - https://github.com/nlpyang/BertSum

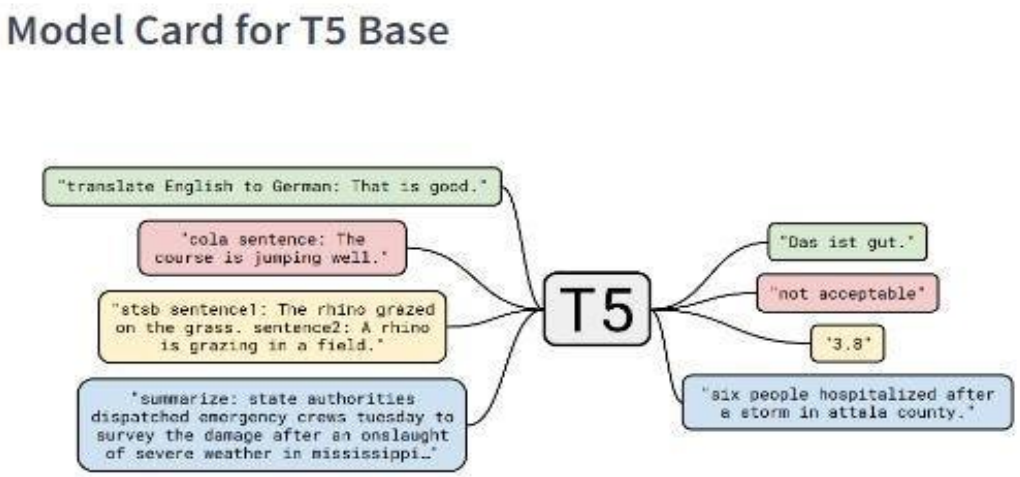
🔹 **Sample Input (Document):**

> The Eiffel Tower is one of the most recognizable landmarks in the world. It was built in 1889 for the Exposition Universelle held to celebrate the 100th anniversary of the French Revolution. Designed by Gustave Eiffel, the tower was initially criticized by many of France's leading artists and intellectuals for its design. However, it has become a global cultural icon of France and one of the most-visited monuments in the world. The Eiffel Tower stands 300 meters tall and was the tallest man-made structure in the world for 41 years.

🔹 **Sample Output (Extractive Summary):**

> The Eiffel Tower is one of the most recognizable landmarks in the world.
> It has become a global cultural icon of France and one of the most-visited monuments in the world.

Abstractive Summarization Models



- **BART (Bidirectional and Auto-Regressive Transformers)**:
  - a transformer model that uses a denoising autoencoder for pretraining. It's highly effective for both extractive and abstractive summarization tasks by learning to reconstruct corrupted versions of input text. BART has been shown to outperform earlier models like Seq2Seq with attention in generating high-quality summaries.
- **T5 (Text-to-Text Transfer Transformer)**:
  - a unified framework that treats every NLP task, including summarization, as a text-to-text problem. T5 is pre-trained on a large amount of text and then fine-tuned for specific tasks like summarization. It generates human-like summaries and can handle a wide range of document types.
- **GPT (Generative Pre-trained Transformer)**
  - autoregressive transformers that generate text one token at a time. GPT can be used for abstractive summarization by conditioning on the input text and generating coherent, contextually relevant summaries.

Hybrid Models (Extractive + Abstractive)

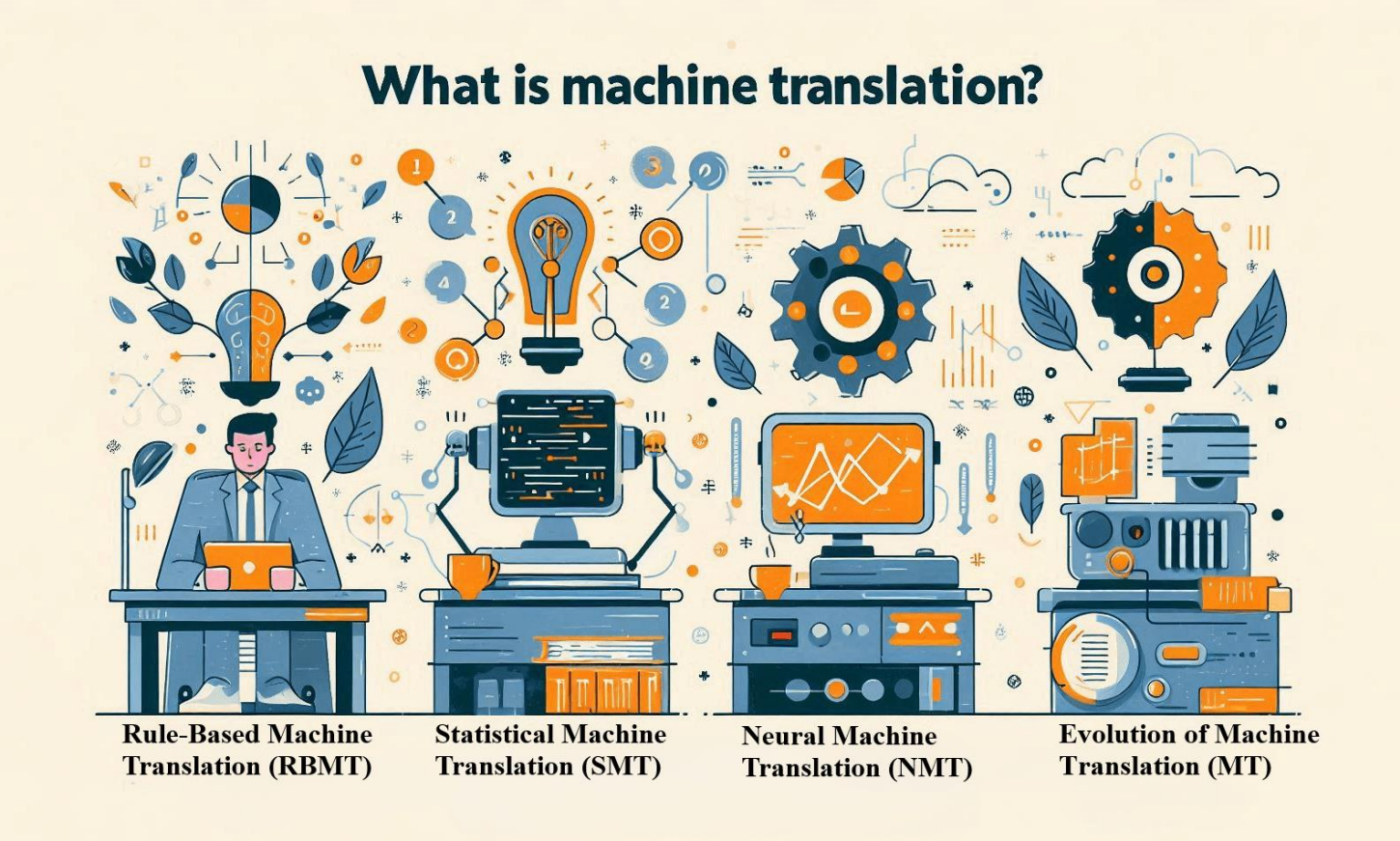
- **Text Preprocessing**:
  - Tokenization, sentence splitting, and stopword removal.
- **Extractive Summarization**:
  - Rank sentences using techniques like TextRank, TF-IDF, or BERT embeddings.
  - Select a subset of the most relevant sentences.
- **Abstractive Summarization**:
  - Input the extracted sentences into a sequence-to-sequence model (e.g., T5, BART, or Pegasus).
  - Generate a coherent and concise summary.
- **Advantages**
  - **Better Coherence**: Combines the contextual relevance of extractive summarization with the fluency of abstractive methods.
  - **Handles Long Documents**: The extractive component ensures that only the most relevant information is processed, making it scalable to longer texts.
  - **Domain Adaptability**: Useful in domains like medical or legal summarization, where factual accuracy is critical.

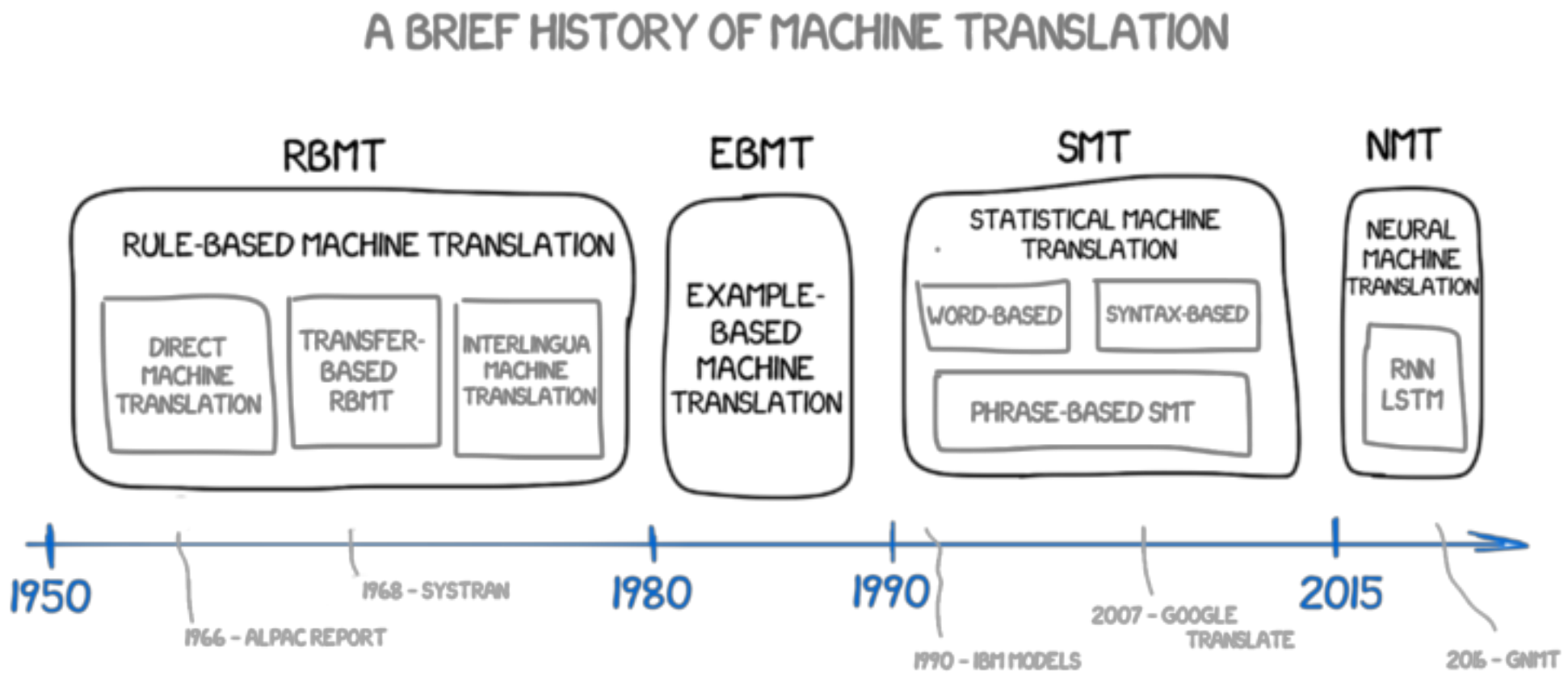
Challenges in Summarization

- **Preserving Meaning**
  - Retains the original meaning of the text while being concise. Abstractive models, in particular, may omit important information or introduce incorrect information due to the complexity of text generation.
- **Coherence and Fluency**
- **Handling Long Texts**:
  - Both extractive and abstractive summarization models often face difficulties when summarizing very long documents, as they may lose important context or be unable to process the entire document effectively.
- **Domain-Specific Summarization**:
  - Summarizing texts in specialized domains (e.g., legal, medical, or scientific texts) requires domain-specific models, as general-purpose summarization models may not effectively capture key concepts from these domains.

Machine Translation

- The task of automatically translating text or speech from one language to another while preserving the meaning, context, and fluency. It is a key NLP application used in communication, international business, education, and more.




A BRIEF HISTORY OF MACHINE TRANSLATION
RBMT
RULE-BASED MACHINE TRANSLATION
DIRECT MACHINE TRANSLATION
TRANSFER-BASED RBMT
INTERLINGUA MACHINE TRANSLATION
EBMT
EXAMPLE-BASED MACHINE TRANSLATION
SMT
STATISTICAL MACHINE TRANSLATION
WORD-BASED
SYNTAX-BASED
PHRASE-BASED SMT
NMT
NEURAL MACHINE TRANSLATION
RNN
LSTM
1950
1980
1990
2015
1966 - ALPAC REPORT
1968 - SYSTRAN
1990 - IBM MODELS
2007 - GOOGLE TRANSLATE
2016 - GNMT

Rule-Based Machine Translation (RBMT)

- Based on linguistic rules and grammars specific to the source and target languages. Translates by applying a set of hand-crafted rules for word mappings, syntax, and morphology.
- Example: SYSTRAN, https://en.wikipedia.org/wiki/SYSTRAN
- **Pros**:
  - Interpretable and customizable.
  - Handles specific domains well if sufficient linguistic rules are defined.
- **Cons**:
  - Requires extensive linguistic knowledge.
  - Struggles with idiomatic expressions and creative language.

Statistical Machine Translation (SMT)

- Uses statistical methods to learn mappings between languages from parallel corpora.
  - Example: Google Translate (pre-2016).
- **Key Techniques**:
  - Phrase-Based Models: Translate text by breaking it into phrases.
  - Word Alignment: Determines which words in the source align with words in the target.
  - Language Models: Ensure fluency in the target language.
- **Pros**:
  - Data-driven and less reliant on linguistic rules.
  - Can handle domain adaptation with large domain-specific corpora.
- **Cons**:
  - Requires a lot of parallel data.
  - Poor at capturing long-range dependencies or context.

Neural Machine Translation (NMT)

- Leverages neural networks for end-to-end translation.
- Has replaced SMT in most modern systems due to superior performance.
- **Key Techniques in NMT**:
  - **Encoder-Decoder Framework**:
    Encodes the source sentence into a fixed representation and decodes it into the target language.
  - **Attention Mechanism**:
    Allows the model to focus on specific parts of the source sentence during translation.
  - **Transformers**:
    Replaced recurrent models with self-attention, enabling parallel processing and improved scalability.

Neural Machine Translation (NMT) Models

- **Seq2Seq with Attention**: Early NMT models.
- **Transformer-Based Models**: BERT, GPT, T5, etc.
- **Google Neural Machine Translation (GNMT)**: Uses NMT with advanced attention mechanisms.
- **MarianNMT**: An efficient framework for NMT, widely used for research and industry.

Transformer-Based Models

- **BERT (Bidirectional Encoder Representations from Transformers)**: Pre-trained for understanding tasks, not direct translation, but fine-tuned for MT tasks (e.g., multilingual BERT).
- **mBERT (Multilingual BERT)**: Supports multiple languages simultaneously.
- **GPT (Generative Pre-trained Transformer)**: Works well for text generation and translation tasks.
- **T5 (Text-to-Text Transfer Transformer)**: Treats translation as a text-to-text problem.
- **XLM-R (Cross-lingual Language Model - RoBERTa)**: Designed for multilingual tasks, including translation.
- **M2M-100 (Many-to-Many by Facebook)**: A multilingual model capable of translating directly between 100 languages.

Challenges in Machine Translation

- **Low-Resource Languages**
  - Many languages lack sufficient parallel corpora for training models, making translation challenging.
- **Idiomatic Expressions**
  - Translating idioms, metaphors, and culturally specific phrases requires deep contextual understanding.
- **Long Sentences**
  - Maintaining coherence and accuracy in long or complex sentences is difficult.
- **Domain Adaptation**
  - Models trained on general corpora may struggle with technical or domain-specific texts (e.g., legal or medical).
- **Context Preservation**
  - Handling ambiguity and preserving context, especially in dialogue or conversational scenarios, is a challenge.

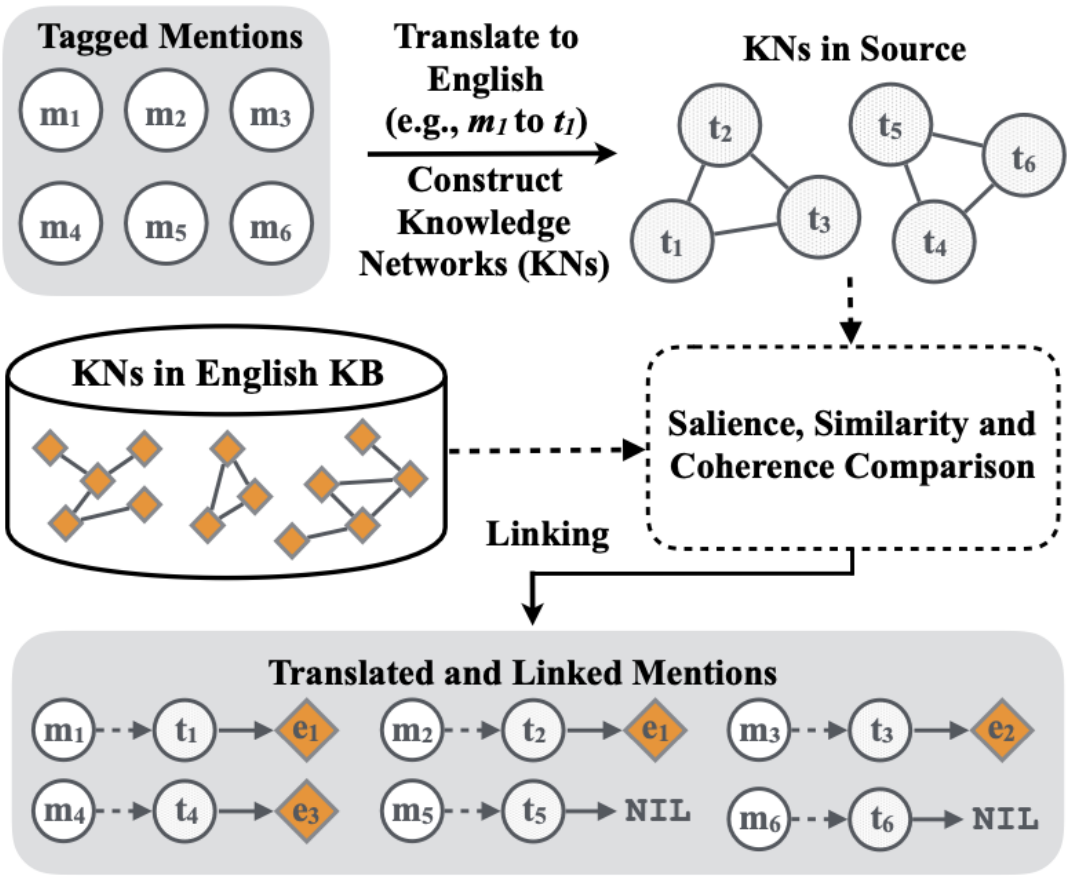


Figure 3: Cross-lingual Entity Linking Overview

Cross-lingual Name Tagging and Linking for 282 Languages (Pan et al., ACL 2017)

Applications of Machine Translation

- **Global Communication**: Breaking language barriers in real-time communication (e.g., Google Translate, Skype Translator).
- **Content Localization**: Translating websites, documents, and applications for international markets.
- **Cross-Language Information Retrieval**: Enabling users to search and retrieve information in other languages.
- **Customer Support**: Assisting customers in multiple languages using automated systems.
- **E-Learning**: Providing multilingual content for education platforms.

Future Directions

- **Few-Shot and Zero-Shot Translation**:
  Leveraging models like GPT-4 or mT5 to translate between languages without direct training on specific language pairs.
- **Multilingual and Universal Models**:
  Enhancing many-to-many translation capabilities with unified multilingual models like **M2M-100** or **XLM-R**.
- **Context-Aware Translation**:
  Incorporating dialogue and document-level context to improve translation coherence and accuracy.
- **Speech-to-Speech Translation**:
  Real-time translation of spoken language, combining speech recognition, MT, and speech synthesis.
- **Ethics and Fairness**:
  Ensuring bias-free translations and addressing ethical concerns in cross-cultural contexts.

Question Answering

- **Definition:** Question Answering (QA) is a subfield of NLP focused on building systems that automatically answer questions posed in natural language.
- **Examples:**
  - **Fact-based:** "Who is the president of France?"
  - **Opinion-based:** "What are the benefits of renewable energy?"
  - **Complex reasoning:** "What caused World War II?"

Types of QA Systems

1. **Open-Domain QA**
    1. Answers questions using unstructured information, often from the web.
    2. Example: Google Search.
2. **Closed-Domain QA**
    1. Operates within a specific domain (e.g., medical, legal).
    2. Example: IBM Watson for healthcare.
3. **Generative vs Extractive QA**
    1. **Extractive:** Answers are extracted directly from the source text.
    2. **Generative:** Answers are synthesized or paraphrased.

Workflow of a QA System

1. **Question Understanding**
    1. Identify question type (e.g., "Who", "What", "Why").
    2. Semantic parsing.
2. **Information Retrieval**
    1. Retrieve relevant documents or passages from a corpus.
3. **Answer Extraction/Generation**

    Use NLP models to extract or generate an answer from the retrieved information.

QA Models

- **Traditional Approaches:**
  - **Rule-Based Systems:** Hand-crafted rules for specific queries.
  - **Information Retrieval (IR) Systems:**
    - Example: TF-IDF, BM25 for passage retrieval.
- **Modern Approaches:**
  - **Extractive Models:**
    - Fine-tuned BERT or DistilBERT models.
    - Example: SQuAD dataset.
  - **Generative Models:**
    - GPT-3, T5, or BART models.
    - Example: Generating long-form answers.
  - **Retrieval-Augmented Models:**
    - Example: RAG (Retrieval-Augmented Generation).
    - Combines retrieval and generative capabilities.

Experiments in QA

- **Datasets for QA**
  - **SQuAD (Stanford Question Answering Dataset):**
    - Extractive QA from Wikipedia.
  - **Natural Questions (Google):**
    - Open-domain QA with real queries.
  - **HybridQA and OTT QA**
  - **AIT-QA and TAT-QA**
- **Evaluation Metrics**
  - **E**xact Match (EM): Proportion of answers exactly matching the ground truth.
  - F1 Score: Weighted average of precision and recall.
  - BLEU/ROUGE Scores: Evaluates the overlap between generated and reference answers.
  - Human Evaluation: Subjective evaluation of fluency, relevance, and accuracy.

Challenges in QA

1. **Ambiguity in Questions**
2. **Answering Multi-Hop Questions:**
    1. Requires combining information from multiple documents.
3. **Domain Adaptation:**
    1. Generalizing across different domains with minimal fine-tuning.
4. **Numbers**
5. **Scale and Latency:**
    1. Handling large corpora efficiently.

QA In Practice

```python
from transformers import pipeline

# Load a pre-trained QA pipeline
qa_pipeline = pipeline("question-answering")

# Example question and context
context = "Natural language processing enables machines to
question = "What does NLP enable?"

# Get the answer
result = qa_pipeline(question=question, context=context)
print(result["answer"])
```

Output: "machines to understand human language"

- Context: "Natural language processing enables machines to understand human language. It is a subset of AI."

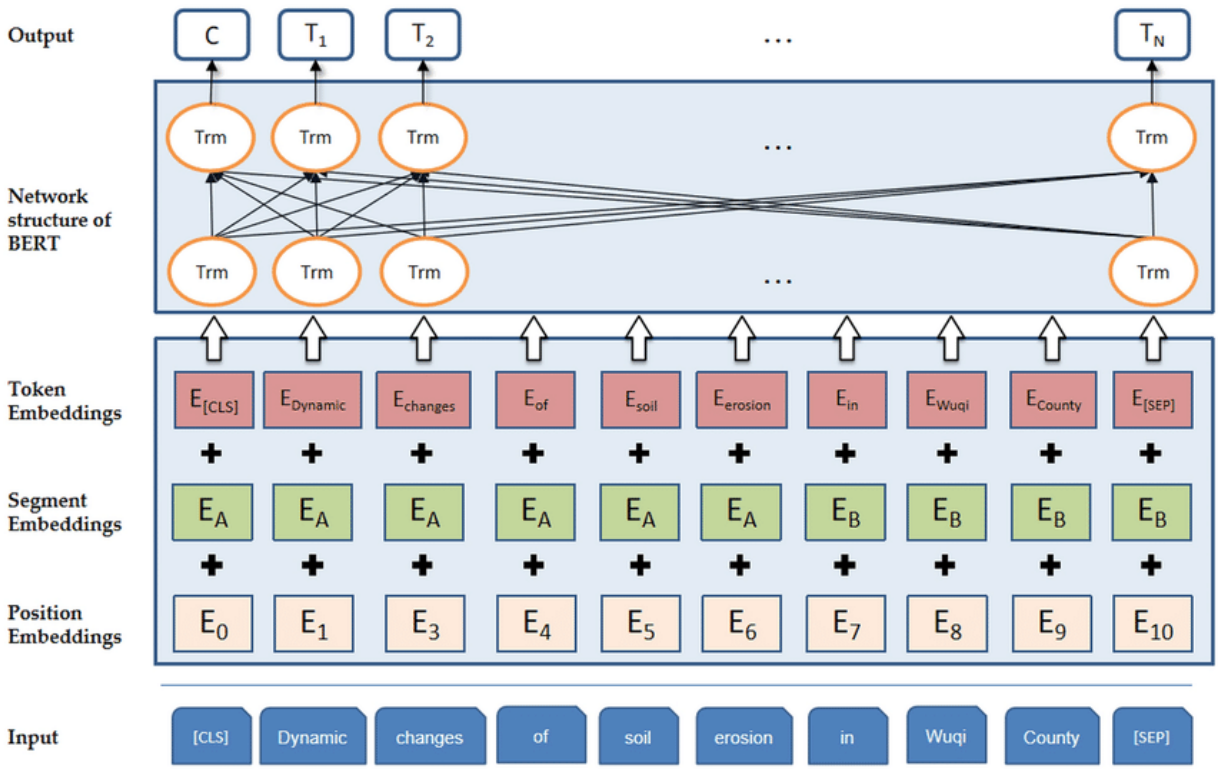
Advanced Tasks and Emerging Topics in NLP

Complex tasks requiring sophisticated techniques and large-scale data.

Introduction to Advanced NLP

- **Traditional NLP vs. Modern NLP**:
  - Shift from rule-based systems → statistical models → neural networks → foundation models (GPT-4, PaLM, LLaMA).
- **Key Drivers of Innovation**:
  - Compute power, large datasets, cross-disciplinary applications.
- **Why Focus on Emerging Tasks?**:
  - Solve unsolved problems (e.g., context-aware reasoning, low-resource languages).
  - Address societal needs (e.g., bias mitigation, climate NLP).

Pre-trained Language Models (PLMs)

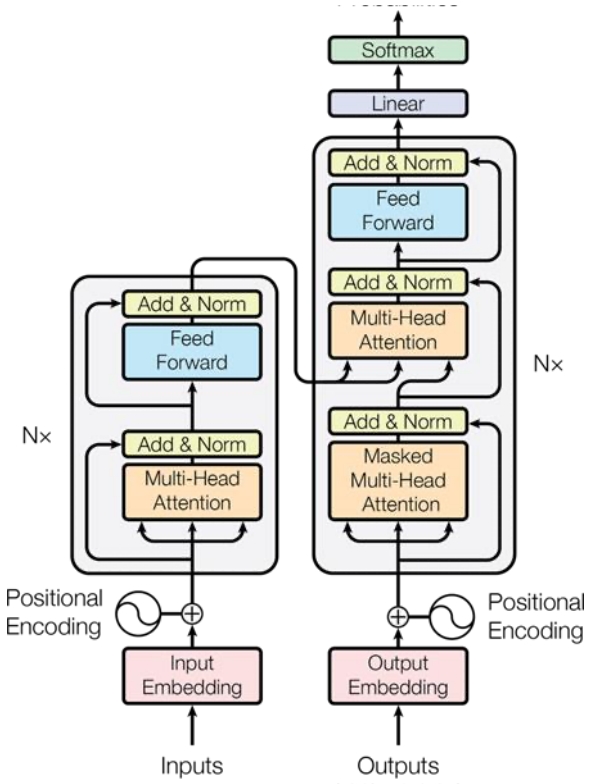


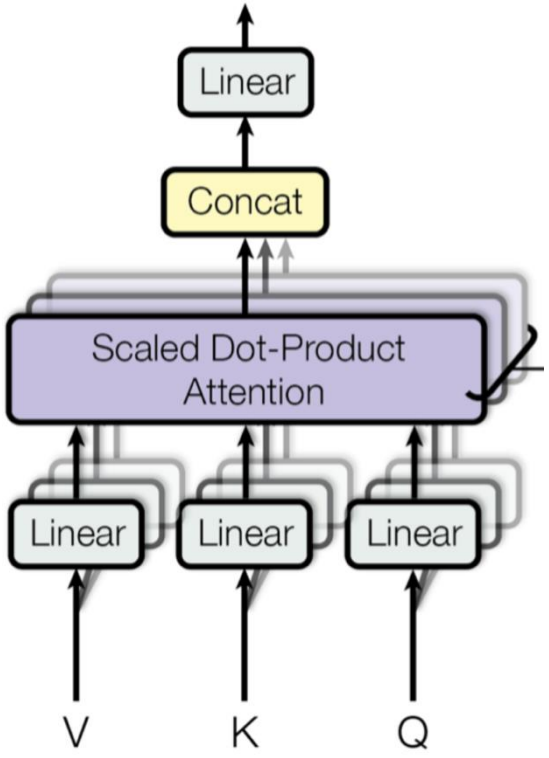
- **What are PLMs?**
  - Pre-trained Language Models are neural networks trained on large text corpora.
  - Fine-tuned for specific downstream tasks like sentiment analysis, machine translation, and question answering.
- **Why are PLMs Important?**
  - They reduce the need for task-specific training data.
  - Enable rapid development of state-of-the-art NLP systems.

Key Milestones in PLM Development

- **Early Approaches:**
  - Word2Vec (2013): Distributed word embeddings.
  - GloVe (2014): Global Vectors for word representation.
- **Transformer Revolution:**
  - Attention is All You Need (2017): Introduction of the Transformer architecture.
- **Modern PLMs:**
  - BERT (2018): Bidirectional Encoder Representations from Transformers.
  - GPT Series (2018 - Present): Generative Pre-trained Transformers.
  - T5 (2020): Text-to-Text Transfer Transformer.

The Transformer Architecture



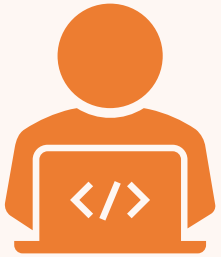


- **Core Components:**
  - **Self-Attention Mechanism:** Captures dependencies regardless of distance in the text.
  - **Positional Encoding:** Preserves the order of words in sequences.
  - **Feed-Forward Networks:** Processes attention outputs.
- **Advantages:**
  - Parallelization through multi-head attention.
  - Scales efficiently with data.

Pre-training and Fine-tuning

- **Pre-training:**
  - Objective: Train on massive unlabeled text corpora.
  - Common Techniques:
    - Masked Language Modeling (e.g., BERT).
    - Causal Language Modeling (e.g., GPT).
- **Fine-tuning:**
  - Tailors the pre-trained model to a specific task.
  - Requires relatively small labeled datasets.

Popular PLMs and Applications

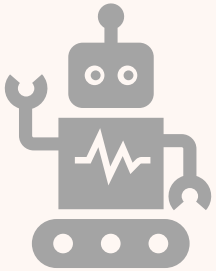


Popular PLMs:

BERT: Used for text classification, named entity recognition.

GPT: Excellent for text generation.

T5: Universal framework for NLP tasks.



Applications:

Sentiment Analysis

Machine Translation

Chatbots and Virtual Assistants

Summarization

NLP for Structured and Semi-Structured Data

- **What is Structured and Semi-Structured Data?**
  - **Structured Data:** Data organized into tables or predefined schemas (e.g., databases, CSV files).
  - **Semi-Structured Data:** Data with partial organization, lacking fixed schema (e.g., JSON, XML, log files).
- **Why Use NLP for These Data Types?**
  - Unlock insights from textual information within structured contexts.
  - Enhance search, analysis, and decision-making processes.

Related NLP Techniques

- **Entity Recognition:**
  - Identifying key entities in tabular data.
- **Semantic Parsing:**
  - Translating natural language queries into formal languages (e.g., SQL).
- **Text Classification:**
  - Categorizing textual fields in structured records.
- **Information Retrieval:**
  - Enabling search and retrieval of relevant entries based on text queries.
- **Schema Parsing:**
  - Extracting structure from formats like JSON and XML.
- **Named Entity Recognition (NER):**
  - Identifying entities within text in semi-structured data.
- **Summarization**
  - Condensing textual content within semi-structured formats.

Tools and Frameworks

- **Popular Models:**
  - TAPAS: Transformer-based model for tabular QA, need pretraining
  - RCI: Transformer-based model for tabular QA, no task specific pretraining
- **For Semi-Structured Data:**
  - SpaCy: Flexible parsing for semi-structured text.
  - Apache Spark NLP: Scalable processing.

Multimodal NLP Overview

- **What is Multimodal NLP?**
  - Combines natural language processing with other data modalities like images, audio, and structured data.
  - Aims to create systems that understand and generate content across multiple types of inputs.
- **Why Multimodal NLP?**
  - Many real-world tasks involve multiple data types (e.g., videos with subtitles, e-commerce product descriptions with images).
  - Provides richer context and better understanding.

Key Modalities in Multimodal NLP

- **Text and Vision:**
  - Images, videos, and associated textual descriptions.
  - Applications: Image captioning, visual question answering (VQA).
- **Text and Audio:**
  - Speech and text-based transcriptions.
  - Applications: Speech recognition, voice assistants.
- **Text and Structured Data:**
  - Tabular data, databases with textual annotations.
  - Applications: Data-to-text generation, multimodal question answering.
- **Text, Vision, and Audio:**
  - Integrating multiple modalities simultaneously.
  - Applications: Multimodal emotion detection, multimedia content summarization.

Key Challenges in Multimodal NLP

- **Representation Learning:**
  - Encoding different modalities into a shared feature space.
- **Alignment:**
  - Matching information across modalities (e.g., linking image regions with text phrases).
- **Fusion:**
  - Combining features from multiple modalities effectively.
- **Scalability:**
  - Handling large and diverse multimodal datasets.
- **Ambiguity Resolution:**
  - Resolving conflicts or ambiguities when modalities provide contradictory information.

Techniques and Architectures

- **Early Fusion:**
  - Combine raw data from all modalities at the input level.
  - Example: Concatenating text and image embeddings.
- **Late Fusion:**
  - Process each modality separately and combine outputs.
  - Example: Combining predictions from text and image classifiers.
- **Cross-Attention Mechanisms:**
  - Use attention layers to focus on relevant parts of each modality.
  - Example: Linking image regions to specific words in captions.
- **Pretrained Multimodal Models:**
  - CLIP (Contrastive Language-Image Pretraining): Links text and images.
  - Flamingo: Handles text, vision, and other modalities.

Applications of Multimodal NLP

- **Image Captioning:**
  - Generate textual descriptions for images.
- **Visual Question Answering (VQA):**
  - Answer questions about an image using both text and visual inputs.
- **Multimodal Search Engines:**
  - Search using combinations of text and images.
- **E-commerce:**
  - Enhance product search and recommendations using images, text, and user reviews.
- **Social Media Analysis:**
  - Analyze multimedia posts for sentiment, trends, and misinformation.
- **Healthcare:**
  - Combine medical imaging with patient records for diagnosis support.

Let’s Look at the Famous CLIP

- **Overview:**
  - CLIP is designed to learn **joint representations** of images and text, enabling tasks like zero-shot image classification, cross-modal retrieval, and image generation guidance (e.g., in DALL·E).
- **Key Idea**:
  - Train a model to predict which **text descriptions** correspond to which **images** from a large dataset of (image, text) pairs.
  - Use contrastive learning to align image and text embeddings in a shared semantic space.
- **Key Features:**
  - Zero-shot learning: Performs tasks without task-specific training.
  - Applications: Image classification, multimodal search.
- **Impact:**
  - Improved performance on diverse tasks without requiring fine-tuning.

CLIP Architecture

- CLIP consists of **two parallel encoders**:
  - **Image Encoder** (e.g., Vision Transformer or ResNet) → Encodes images into embeddings.
  - **Text Encoder** (Transformer-based) → Encodes text into embeddings.

  [Input Image] → [Image Encoder] → Image Embedding

  [Input Text] → [Text Encoder] → Text Embedding

CLIP is trained on **400 million (image, text) pairs** from the internet.

- **Contrastive Learning Workflow:**
  - **Batch Processing**:
    - A batch of N images and N text captions is fed into the encoders.
    - Each image is paired with one correct text (and N-1 incorrect pairs).
  - **Similarity Matrix**:
    - Compute a similarity matrix where element (i, j) is the cosine similarity between the i-th image embedding and j-th text embedding.
  - **Loss Function**:
    - **Contrastive Loss**: Maximize similarity for correct pairs (diagonal) and minimize it for incorrect pairs.

Zero-Shot Inference

- CLIP enables **zero-shot classification** without task-specific fine-tuning:
  - **Text Prompts**: Define class labels as text prompts (e.g., *"a photo of a {dog, cat, car}"*).
  - **Compare Embeddings**: Compute similarity between the image embedding and all text embeddings.
  - **Predict**: Select the text prompt with the highest similarity score.

[Input Image] → [Image Encoder] → Image Embedding
↘
[Text Prompts] → [Text Encoder] → Text Embeddings → [Cosine Similarity] → Prediction

CLIP in Hugging face

```python
import requests
from transformers import CLIPProcessor, CLIPModel

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# Load image and define labels
image = Image.open("cat.jpg")
labels = ["a photo of a cat", "a photo of a dog", "a photo of a car"]

# Process inputs
inputs = processor(text=labels, images=image, return_tensors="pt", padding=True)
outputs = model(**inputs)
logits_per_image = outputs.logits_per_image  # Similarity scores

# Get predicted label
predicted_idx = logits_per_image.argmax().item()
print(f"Predicted: {labels[predicted_idx]}")
```

Emerging Topics in NLP

Multimodal Learning

Combining text, images, and other data types.

Example: Visual Question Answering (VQA).

Pre-trained Language Models

Innovations beyond BERT: GPT, T5, LLaMA.

Few-Shot and Zero-Shot Learning

Solving tasks with minimal or no labeled data.

Responsible AI in NLP

Addressing bias, fairness, and explainability.

Neuromorphic NLP

Mimicking human-like learning processes

Assignment 2

- NER in Old days
- Upload to Canvas tonight
- Due in 2 weeks
- Grades for assignment 1 will be back to you in a week